

Gestion d'une boutique



Table des matières

	Pages
Table des matières	2
Historique du document	3
Description générale du projet	4-5
<i>Contexte du projet</i>	4
<i>Domaine étudié</i>	5
<i>Fonctionnement du système</i>	5
<i>Périmètre du système</i>	5
<i>Extensions possibles</i>	5
Conception du projet	
<i>Dictionnaire de données</i>	6-8
<i>Explication des associations et des cardinalités</i>	8-10
<i>Modèle logique de données relationnel normalisé</i>	10-11
<i>Explication de la normalisation</i>	11
<i>Script de création de la base de données</i>	12-17
<i>Diagramme de contexte</i>	17
<i>Diagramme de cas d'utilisation</i>	18-21
<i>Diagramme de séquence</i>	22
<i>Diagramme de classes</i>	23
Finalisation du projet	
<i>IHM</i>	24-27
<i>Principales requêtes développées</i>	28
<i>Quelques exemples de codes</i>	29-31
<i>Conclusion</i>	31

Historique du document

Cette partie permettra d'identifier les différentes phases importantes du projet, elle sera donc constamment mise à jour en fonction de l'avancement de celui-ci.

Version	Date de mise à jour	Auteur	Modifications essentielles
V1	22/11/2012	GENEY Adrien et TARNAUD Florian	Description du contexte
V2	03/12/2012	GENEY Adrien Et TARNAUD Florian	Conception du projet (MCD, MLD, Script de BDD et UML)
V3	11/01/2013	GENEY Adrien Et TARNAUD Florian	Finalisation du projet (IHM, requêtes et code)

Description du sujet de projet

Contexte du projet

Identification et activité

Une nouvelle boutique intitulée « Ellemebotte » a fait son apparition au centre-ville de Besançon depuis le mois de juillet 2012 à l'adresse suivante :

32 rue Bersot
25000 BESANCON

L'activité de cette nouvelle entreprise est la vente de chaussures et d'accessoires de mode (sacs à main, bijoux...). La boutique est ouverte du mardi au samedi de 10h à 19h non-stop.

Clientèle et concurrence

Sa clientèle est très variée, elle concerne les hommes et les femmes de tout âge, cependant le cœur de sa clientèle reste les femmes de moins de 60ans.

En ce qui concerne le marché, on remarque que celui de la chaussure sur la ville de Besançon est basé sur une concurrence pure et parfaite car on trouve un nombre important de vendeurs et d'acheteurs.

La concurrence directe de cette boutique se compose des divers magasins de chaussures du centre-ville de Besançon.

Sa concurrence indirecte est variée car elle est formée de supermarchés et d'hypermarchés qui proposent quelques modèles de chaussures mais également des sociétés e-commerces ayant la même activité (Zalando, Sarenza...).

Statut, produits et fournisseurs

La gérante de la boutique qui est l'unique salariée a opté pour le statut d'auto-entrepreneur.

Au niveau de sa gamme de produits, on peut noter qu'elle se décompose ainsi :

- Chaussures pour femmes en grande quantité : talons, bottes, ballerines, tongs...
- Chaussures pour hommes : baskets, mocassins, chaussures de ville...
- Sac à mains
- Bijoux

Les chaussures varient en fonction des saisons, les prix de celles-ci vont de 15€ à plus de 200€.

Les fournisseurs sont nombreux, on y trouve de très grandes marques (Temps des Cerises, US Marshall, Mustang...) mais également de petites et nouvelles marques (Maria Mare, Lollipops, Poti Pati...).

Motivations de l'entreprise à notre égard

Cet auto-entrepreneur nous a contacté pour palier un problème présent dans toute entreprise : la gestion du stock.

Ces attentes sont claires et concises, elle souhaite pouvoir : gérer son stock (entrées et sorties), ses clients et ses fournisseurs.

Domaine étudié

Domaine de l'entreprise

L'entreprise appartient au domaine marchand car sa principale activité est la vente de produits. La gérante possède un point de vente réel de 30m², de plus elle proposera au cours de l'année 2013 un site internet marchand qui lui permettra d'obtenir un point de vente virtuel.

Domaine de développement

Afin de développer l'application, nous allons nous orienter sur deux langages que nous maîtrisons :

- Le C# : Langage orienté objet
- Mysql : SGBD (Système de Gestion de Base de Données)

Nous avons utilisé le C# car c'est un langage avec lequel nous avons une certaine expérience grâce aux multiples travaux déjà réalisés.

En ce qui concerne Mysql, nous avons préféré ce SGBD plutôt qu'un autre car vu que la gérante va proposer un site internet d'ici peu, l'application que nous allons créer et le site devront dialoguer avec la même base de données, de ce fait PHP et Mysql forme un couple idéal, ce qui explique notre choix.

Fonctionnement du système

Notre programme sera une application qui devra être utilisée à partir d'un micro-ordinateur, ses fonctionnalités seront comprises dans les modules suivants:

- CLIENT : permettra l'ajout d'un nouveau client et la recherche d'informations sur un client existant
- VENTES : gèrera les ventes auprès des clients effectués en boutique
- FOURNISSEUR : offrira la possibilité d'ajouter un nouveau fournisseur ou de rechercher des informations sur un fournisseur existant
- PRODUIT : permettra l'ajout d'un nouveau produit dans le stock, d'obtenir des informations sur un produit et d'en vérifier la disponibilité
- APPROVISIONNEMENT : contrôlera les entrées de produits du stock

Périmètre du système

Notre application permettra de réaliser les diverses opérations citées ci-dessus (gestion des clients, des fournisseurs, des produits, des ventes et des approvisionnements). D'autres fonctionnalités pourront être ajoutées à l'application cependant elles seront le fruit d'extensions supplémentaires monnayées.

Extensions

Les extensions possibles et envisageables sont les suivantes :

- Création d'un module 'Statistiques' qui permettrait d'obtenir diverses informations : produit le mieux ou le moins bien vendu sur une période, client le plus important, panier moyen des clients, fournisseur avec lequel on effectue le plus de transactions...

Conception du projet

Dictionnaire de données

Nom Conceptuel	Nom Logique	Type (E, Ca, Co)	Nature (N, A, AN)	Longueur	Identifiant	Exemple de valeur
Référence du Client	Reference_Client	E	AN	17	Oui	CLT124589DF21201235
Nom du Client	Nom_Client	E	A	50		Dupont
Prénom du client	Prenom_Client	E	A	30		Jean
Date de naissance du client	DateNaissance_Client	E	Date	10		30/10/1985
Adresse du client	Adresse_Client	Co	AN	100		12 rue des vosges
Téléphone Fixe du Client	TelephoneFixe_Client	E	N	10		0384256351
Telephone Portable du Client	TelephonePortable_Client	E	N	10		0615131555
Mail du client	Mail_Client	E	AN	50		robert.jean@gmail.com
Sexe du client	Sexe_Client	E	A	20		Homme
Date d'inscription du client	DateInscription_Client	E	Date	10		21/11/2012
Identifiant du Produit	Id_produit	E	N	10	Oui	365
Reference du produit	Reference_Produit	E	AN	20		BOT125
Libellé du produit	Libelle_Produit	E	A	50		Botte en cuir
Description du produit	Description_Produit	E	A	200		Paire de botte en cuir noir avec semelle compensée
Prix d'achat TTC du produit	PrixAchatTTC_Produit	E	N	10		30
Prix de Vente TTC du produit	PrixVenteTTC_Produit	E	N	10		49,99
Référence du fournisseur	Reference_Fournisseur	E	AN	17	Oui	FRN214587Lk45201214
Nom du fournisseur	Nom_Fournisseur	E	A	50		Cordonnerie25
Adresse du fournisseur	Adresse_Fournisseur	Co	AN	100		15 grande place
Téléphone du fournisseur	TelephoneFixe_Fournisseur	E	N	10		0381355814
Mail du fournisseur	Mail_Fournisseur	E	A	50		Cordonnerie25@gmail.com
Référence de la commande	Reference_Commande	E	AN	20	Oui	COM146521LF14201235
Date de la commande	Date_Commande	E	Date	10		15/11/2012
Quantité totale de la commande	QuantiteTotale_Commande	Ca	N	5		8
Remise totale sur la commande	RemiseTotale_Commande	Ca	N	10		26,5
Montant total de la commande	MontantTotal_Commande	Ca	N	10		150
Référence de la marque	Reference_Marque	E	AN	10	Oui	MAR3
Nom de la marque	Libelle_Marque	E	A	50		Nike
Référence de la sous-catégorie	Reference_SousCategorie	E	AN	10	Oui	SCAT1
Nom de la sous-catégorie	Libelle_SousCategorie	E	A	50		Talon aiguille
Référence de la taille	Reference_Taille	E	AN	10	Oui	T4
Libellé de la taille	Libelle_Taille	E	A	20		42

Référence de la couleur	Reference_couleur	E	AN	10		COUL6
Libellé de la couleur	Libelle_Couleur	E	A	30		Bleu
Référence du représentant du fournisseur	Reference_RepresentantFournisseur	E	AN	10	Oui	RFN124589LF14201225
Nom du représentant du fournisseur	Nom_RepresentantFournisseur	E	A	50		Richemont
Prénom du représentant du fournisseur	Prenom_RepresentantFournisseur	E	A	30		Pierre
Téléphone du représentant	Telephone_RepresentantFournisseur	E	N	10		0645563185
Mail du représentant	Mail_RepresentantFournisseur	E	AN	50		dupont.pierre@cordonneur25.fr
Identifiant de la TVA	Id_TVA	E	N	2	Oui	2
Code de la TVA	Code_TVA	E	N	2		3
Taux de TVA	Taux_TVA	E	N	4		19,6
Date de commencement de la TVA	DateDebut_TVA	E	Date	10		01/04/2000
Date de changement de la TVA	DateFin_TVA	E	Date	10		01/04/2013
Référence de la catégorie	Reference_Categorie	E	AN	10	Oui	CAT4
Nom de la catégorie	Libelle_Categorie	E	A	50		Chaussure
Référence de la ligne de stock	Reference_LigneStock	E	N	30	Oui	754
Date de mouvement du produit	DateMouvement_LigneStock	E	Date	10		22/11/2012
Quantité de mouvement du produit	QuantiteMouvement_LigneStock	E	N	5		2
Quantité disponible du produit	QuantiteDispo_Stock	Ca	N	5		40
Référence du type mouvement	Reference_TypeMouvement	E	N	2	Oui	6
Libellé du Type de mouvement	Libelle_TypeMouvement	E	A	30		Achat
Entrée/sortie	Sens_TypeMouvement	E	A	10		Entrée
Référence de l'approvisionnement	Reference_Approvisionnement	E	AN	20	Oui	145
Date de l'approvisionnement	Date_Approvisionnement	E	date	10		22/11/2012
Quantité totale de l'approvisionnement	QuantiteTotale_Approvisionnement	Ca	N	5		50
Remise totale sur l'approvisionnement	RemiseTotale_Approvisionnement	Ca	N	10		20
Montant total de l'approvisionnement	MontantTotal_Approvisionnement	Ca	N	10		150
Référence de la ligne de commande	Reference_LigneCommande	E	N	30	Oui	123
Remise sur la ligne de commande	Remise_LigneCommande	E	N	10		26,5
Quantité de produit sur ligne de commande	Quantite_LigneCommande	E	N	5		8
Montant de la ligne de commande	Montant_LigneCommande	Ca	N	10		150
Référence de la ligne d'approvisionnement	Reference_LigneApprovisionnement	E	N	30	Oui	652
Quantité de produits sur ligne d'approvisionnement	Quantite_LigneApprovisionnement	E	N	5		50
Remise sur la ligne d'approvisionnement	Remise_LigneApprovisionnement	Ca	N	10		20
Montant de la ligne d'approvisionnement	Montant_LigneApprovisionnement	Ca	N	10		150
Référence du mode de paiement	Ref_ModePaiement	E	N	2	Oui	3
Libellé mode de paiement	Libelle_ModePaiement	E	A	20		Carte bancaire

Code postal	CP_CodePostal	E	N	5	Oui	90000
Nom de la ville	Ville_CodePostal	E	A	50	Oui	Belfort

Explication des associations et des cardinalités

Voici une explication par entité des associations et des règles de gestion du modèle conceptuel de données.

CLIENT :

Un client est identifié par une référence et possède les attributs suivants : nom, prénom, date de naissance, adresse (sans la ville et le code postal), numéro de téléphone fixe, numéro de téléphone portable, mail, sexe et date d'inscription.

Un client habite dans 1 seule ville (1,1), il peut ne pas avoir effectué de commande mais également en avoir passé plusieurs (0,n).

CODE POSTAL :

Une ville est identifiée par son nom et par son code postal, elle peut contenir 0 ou plusieurs clients. La situation pour les fournisseurs est la même que celle du client.

COMMANDE :

Une commande est identifiée par une référence, les attributs d'une commande sont les suivants : date, quantité totale de produits commandés, remise totale, montant total. Une commande possède 1 ou plusieurs lignes de commandes (1,n) et elle est effectuée par un et un seul client (1,1).

LIGNE DE COMMANDE :

Une ligne de commande possède les mêmes attributs que la commande sauf la date, elle détient un seul produit (1,1) et est présente dans une seule commande (1,1). On peut noter qu'une ligne de commande ne peut exister sans une commande (identifiant relatif).

PRODUIT :

Un produit est traçable grâce à un identifiant appelé (ID), il contient une référence, un libellé, une description, un prix d'achat TTC et un prix de vente TTC. Un seul taux de TVA est appliqué sur celui-ci (1,1), il peut être d'une marque ou non (0,1), il est d'une seule sous-catégorie (1,1). Un produit possède une ou aucune couleur (0,1) et il est obligatoirement d'une et une seule taille (1,1). Celui-ci peut être compris dans 0 ou plusieurs lignes d'approvisionnement, lignes de commande et lignes de stock (0,n).

TVA :

La TVA est traçable grâce à un identifiant (ID), elle possède un code, un taux, une date de début et une date de fin. Une TVA peut être appliquée à 0 ou plusieurs produits (0,n).

MARQUE :

Une marque est identifiée par une référence et possède un libellé, 0 ou plusieurs produits peuvent avoir la même marque (0,n).

SOUS-CATEGORIE :

Une sous-catégorie contient un libellé (ex : Escarpins, tongs, baskets...) , elle peut concerner 0 ou plusieurs produits (0,n) mais elle appartient obligatoirement à une seule catégorie (1,1).

CATEGORIE :

Une catégorie possède un libellé (ex : Chaussure, Sac à main, Accessoire...), elle possède 0 ou plusieurs sous-catégories (0,n).

COULEUR :

Une couleur contient un libellé (ex : Jaune, Rouge, Vert...), elle peut se trouver dans 0 ou plusieurs produits (0,n).

TAILLE :

Une taille possède un libellé (ex : 38, 39, Unique...), elle peut concerner 0 ou plusieurs produits (0,n).

LIGNE_STOCK :

Les lignes de stock permettent d'enregistrer les différents mouvements du stock. Une ligne de stock est identifiée par une référence, on y trouve la date du mouvement, la quantité liée au mouvement (achat de 5 paires de chaussures par exemple) et la quantité disponible pour ce produit (après l'achat des 5 paires de chaussures il en reste 2 dans le stock). Une ligne de stock est obligatoirement d'un seul type de mouvement (1,1) et concerne 1 et 1 seul produit (1,1).

TYPE_MOUVEMENT :

Un type de mouvement permet d'identifier la nature du mouvement dans le stock, il contient une référence, un libellé (Achat, Vente, Vol, Casse...) et un sens qui permet de savoir s'il s'agit d'une entrée ou d'une sortie de stock. Un type de mouvement s'applique à 0 ou plusieurs lignes de stocks (0,n).

APPROVISIONNEMENT :

Cette entité fonctionne comme celle de 'Commande' sauf qu'ici elle ne concerne pas le client mais le fournisseur, de ce fait un approvisionnement peut contenir 1 ou plusieurs lignes d'approvisionnement (1,n) et un approvisionnement concerne 1 et 1 seul fournisseur (1,1).

LIGNE_APPROVISIONNEMENT :

Cette entité fonctionne comme celle de 'Ligne_Commande' sauf qu'ici elle ne concerne pas une commande mais un approvisionnement. De ce fait une ligne d'approvisionnement contient 1 et 1 seul produit (1,1) et concerne 1 et 1 seul approvisionnement (1,1).

FOURNISSEUR:

Un fournisseur possède les attributs suivants : une référence, un nom, une adresse, un numéro de téléphone fixe et un mail. Un fournisseur peut avoir 0 ou plusieurs représentants (0,n) et il peut avoir effectuer 0 ou plusieurs approvisionnements (0,n).

REPRESENTANT_FOURNISSEUR :

Un représentant fournisseur possède quasiment les mêmes attributs qu'un fournisseur sauf qu'il n'a pas d'adresse et qu'il détient un prénom. Celui-ci ne peut dépendre que de un et un seul fournisseur (1,1).

MODE_PAIEMENT :

Un mode de paiement contient les différents modes de paiements possibles (Carte bancaire, chèque, espèces...). Un mode de paiement concerne 0 ou plusieurs commandes (0,n).

Modèle logique des données relationnel normalisé

CLIENT (Reference_Client, Nom_Client, Prenom_Client, DateNaissance_Client, Adresse_Client, TelephoneFixe_Client, TelephonePortable_Client, Mail_Client, Sexe_Client, DateInscription_Client, # CP_CodePostal, # Ville_CodePostal)

PRODUIT (Id_Produit, Reference_Produit, Libelle_Produit, Description_Produit, PrixAchatTTC_Produit, PrixVenteTTC_Produit, # Reference_Taille, # Reference_Couleur, # Reference_SousCategorie, # Reference_Marque, # Id_TVA)

FOURNISSEUR (Reference_Fournisseur, Nom_Fournisseur, Adresse_Fournisseur, TelephoneFixe_Fournisseur, Mail_Fournisseur, # CP_CodePostal, # Ville_CodePostal)

COMMANDE (Reference_Commande, Date_Commande, QuantiteTotale_Commande, RemiseTotale_Commande, MontantTotal_Commande, # Reference_Client, # Reference_ModePaiement)

MARQUE (Reference_Marque, Libelle_Marque)

SOUS_CATEGORIE (Reference_SousCategorie, Libelle_SousCatégorie, # Reference_Categorie)

TAILLE (Reference_Taille, Libelle_Taille)

COULEUR (Reference_Couleur, Libelle_Couleur)

REPRESENTANT_FOURNISSEUR (Reference_RepresentantFournisseur, Nom_RepresentantFournisseur, Prenom_RepresentantFournisseur, Telephone_RepresentantFournisseur, Mail_RepresentantFournisseur, # Reference_Fournisseur)

TVA (Id_TVA, Code_TVA, Taux_TVA, DateDebut_TVA, DateFin_TVA)

CATEGORIE (Reference_Categorie, Libelle_Catégorie)

LIGNE_STOCK (Reference_LigneStock, DateMouvement_LigneStock, QuantiteMouvement_LigneStock, QuantiteDispo_Stock, # Id_Produit, # Reference_TypeMouvement)

TYPE_MOUVEMENT (Reference_TypeMouvement, Libelle_TypeMouvement, Sens_TypeMouvement)

CODE_POSTAL (CP_CodePostal, Ville_CodePostal)

APPROVISIONNEMENT (Reference_Approvisionnement, Date_Approvisionnement, QuantiteTotale_Approvisionnement, RemiseTotale_Approvisionnement, MontantTotale_Approvisionnement, # Reference_Fournisseur)

LIGNE_COMMANDE (Reference_LigneCommande, Remise_LigneCommande, Quantite_LigneCommande, Montant_LigneCommande, # Id_Produit, # Reference_Commande)

LIGNE_APPROVISIONNEMENT (Reference_LigneApprovisionnement, Quantite_LigneApprovisionnement, Remise_LigneApprovisionnement, Montant_LigneApprovisionnement, # Id_Produit, # Reference_Approvisionnement)

MODE_PAIEMENT (Reference_ModePaiement, Libelle_ModePaiement).

Explications de la normalisation

Nous avons créé les entités TAILLE, COULEUR, MARQUE pour ne pas avoir de doublon sur l'entité PRODUIT qui reprend les caractéristiques d'un produit. De même une entité Mode_Paiement a été créée pour le mode de paiement (chèque, carte bancaire, etc) sur une commande.

Pour connaître la catégorie (chaussure, sac ...) et la sous-catégorie (bottes, bottines, talons ...) d'un produit nous avons relié l'entité PRODUIT à SOUS_CATEGORIE, grâce à cela nous aurons directement la sous-catégorie et indirectement grâce à la clé étrangère Reference_Categorie nous obtiendrons la catégorie car une sous-catégorie ne peut avoir qu'une seule catégorie.

Sur les entités COMMANDE et APPROVISIONNEMENT nous avons rajouté une entité LIGNE_COMMANDE et LIGNE_APPROVISIONNEMENT pour connaître le détail par produit d'une transaction.

On y retrouvera la quantité, le montant et la remise sur le produit acheté. Une commande étant une commande du client au magasin et un approvisionnement une commande du magasin à un fournisseur.

L'entité TVA a été créée car il peut y avoir différents taux de TVA. On a rajouté à celui-ci un code car le taux de la TVA peut varier avec le temps par exemple la TVA avec le code 1 et de taux 19.6% pourra passer à 20% au courant de l'année 2013. Nous avons rajouté deux champs date (début et fin) pour pouvoir faire un historique de ces taux.

Le fournisseur possède un représentant fournisseur car le magasin rentrera plus souvent en contact avec le représentant du fournisseur que le fournisseur lui-même. On crée donc une entité RepresentantFournisseur pour pouvoir enregistrer les données lui correspondant et pouvoir le recontacter plus tard.

TYPE_MOUVEMENT est une entité permettant de connaître par quel moyen le produit est sorti ou est entré dans le stock : un achat, une vente. TYPE_MOUVEMENT est relié à LIGNE_STOCK qui permet après le mouvement d'un produit de connaître le nombre de produit sorti ou entré et la quantité disponible dans le stock.

Script de création de la base de données

/ Création de la base de données ElleMeBotte */*

```
CREATE DATABASE IF NOT EXISTS ElleMeBotte CHARACTER SET 'utf8';
```

/ On utilise la base de données que l'on vient de créer */*

```
USE ElleMeBotte;
```

/ Création des différentes tables de la base */*

```
CREATE TABLE Client
```

```
(  
Reference_Client VARCHAR(17) NOT NULL,  
Nom_Client VARCHAR(50) NULL,  
Prenom_Client VARCHAR(30) NULL,  
DateNaissance_Client DATE NULL,  
Adresse_Client TEXT(100) NULL,  
TelephoneFixe_Client VARCHAR(10) NULL,  
TelephonePortable_Client VARCHAR(10) NULL,  
Mail_Client VARCHAR(50) NULL,  
Sexe_Client VARCHAR(20) NULL,  
DateInscription_Client DATE NULL,  
CONSTRAINT PRIMARY KEY(Reference_Client)  
)  
ENGINE=INNODB;
```

```
CREATE TABLE Fournisseur
```

```
(  
Reference_Fournisseur VARCHAR(17) NOT NULL,  
Nom_Fournisseur VARCHAR(50) NULL,  
Adresse_Fournisseur TEXT(100) NULL,  
TelephoneFixe_Fournisseur VARCHAR(10) NULL,  
Mail_Fournisseur VARCHAR(50) NULL,  
CONSTRAINT PRIMARY KEY(Reference_Fournisseur)  
)  
ENGINE=INNODB;
```

```
CREATE TABLE Taille
```

```
(  
Reference_Taille VARCHAR(10) NOT NULL,  
Libelle_Taille VARCHAR(20) NULL,  
CONSTRAINT PRIMARY KEY(Reference_Taille)  
)  
ENGINE=INNODB;
```

```
CREATE TABLE Categorie
```

```
(  
Reference_Categorie VARCHAR(10) NOT NULL,  
Libelle_Categorie VARCHAR(50) NULL,  
CONSTRAINT PRIMARY KEY(Reference_Categorie)  
)  
ENGINE=INNODB;
```

```

CREATE TABLE Produit
(
Id_Produit INT(10) UNSIGNED NOT NULL AUTO_INCREMENT,
Reference_Produit VARCHAR(20) NOT NULL,
Libelle_Produit VARCHAR(50) NULL,
Description_Produit TEXT(200) NULL,
PrixAchatTTC_Produit DECIMAL(10,2) NULL,
PrixVenteTTC_Produit DECIMAL(10,2) NULL,
CONSTRAINT PRIMARY KEY(Id_Produit)
)
ENGINE=INNODB;

CREATE TABLE Tva
(
Id_TVA SMALLINT(2) UNSIGNED AUTO_INCREMENT NOT NULL,
Code_TVA SMALLINT(2) UNSIGNED NULL,
Taux_TVA DECIMAL (4,2) NULL,
DateDebut_TVA DATE NULL,
DateFin_TVA DATE NULL,
CONSTRAINT PRIMARY KEY(Id_TVA)
)
ENGINE=INNODB;

CREATE TABLE Representant_Fournisseur
(
Reference_RepresentantFournisseur VARCHAR(17) NOT NULL,
Nom_RepresentantFournisseur VARCHAR(50) NULL,
Prenom_RepresentantFournisseur VARCHAR(30) NULL,
Telephone_RepresentantFournisseur VARCHAR(10) NULL,
Mail_RepresentantFournisseur VARCHAR(50) NULL,
CONSTRAINT PRIMARY KEY (Reference_RepresentantFournisseur)
)
ENGINE=INNODB;

CREATE TABLE Sous_Categorie
(
Reference_SousCategorie VARCHAR(10) NOT NULL,
Libelle_SousCategorie VARCHAR(50) NULL,
CONSTRAINT PRIMARY KEY(Reference_SousCategorie)
)
ENGINE=INNODB;

CREATE TABLE Commande
(
Reference_Commande VARCHAR(17) NOT NULL,
Date_Commande DATE NULL,
QuantiteTotale_Commande INT(5) NULL,
RemiseTotale_Commande DECIMAL(10,2) NULL,
MontantTotal_Commande DECIMAL(10,2) NULL,
CONSTRAINT PRIMARY KEY(Reference_Commande)
)
ENGINE=INNODB;

```

```

CREATE TABLE Ligne_Commande
(
Reference_LigneCommande INT(30) UNSIGNED NOT NULL AUTO_INCREMENT,
Remise_LigneCommande DECIMAL(10,2) NULL,
Quantite_LigneCommande INT(5) NULL,
Montant_LigneCommande DECIMAL(10,2) NULL,
CONSTRAINT PRIMARY KEY(Reference_LigneCommande)
)
ENGINE=INNODB;

CREATE TABLE Mode_Paiement
(
Reference_ModePaiement SMALLINT(2) UNSIGNED NOT NULL AUTO_INCREMENT,
Libelle_ModePaiement VARCHAR(20) NULL,
CONSTRAINT PRIMARY KEY(Reference_ModePaiement)
)
ENGINE=INNODB;

CREATE TABLE Approvisionnement
(
Reference_Approvisionnement VARCHAR(20) NOT NULL,
Date_Approvisionnement DATE NULL,
QuantiteTotale_Approvisionnement INT(5) NULL,
RemiseTotale_Approvisionnement DECIMAL(10,2) NULL,
MontantTotale_Approvisionnement DECIMAL(10,2) NULL,
CONSTRAINT PRIMARY KEY(Reference_Approvisionnement)
)
ENGINE=INNODB;

CREATE TABLE Marque
(
Reference_Marque VARCHAR(10) NOT NULL,
Libelle_Marque VARCHAR(50) NULL,
CONSTRAINT PRIMARY KEY(Reference_Marque)
)
ENGINE=INNODB;

CREATE TABLE Ligne_Approvisionnement
(
Reference_LigneApprovisionnement INT(30) UNSIGNED NOT NULL AUTO_INCREMENT,
Quantite_LigneApprovisionnement INT(5) NULL,
Remise_LigneApprovisionnement DECIMAL(10,2) NULL,
Montant_LigneApprovisionnement DECIMAL(10,2) NULL,
CONSTRAINT PRIMARY KEY(Reference_LigneApprovisionnement)
)
ENGINE=INNODB;

CREATE TABLE Code_Postal
(
CP_CodePostal VARCHAR(5) NOT NULL,
Ville_CodePostal VARCHAR(50) NULL,
CONSTRAINT PRIMARY KEY(CP_CodePostal, Ville_CodePostal)
)
ENGINE=INNODB;

```

```

CREATE TABLE Ligne_Stock
(
Reference_LigneStock INT(30) UNSIGNED NOT NULL AUTO_INCREMENT,
DateMouvement_LigneStock DATE NULL,
QuantiteMouvement_LigneStock INT(5) NULL,
QuantiteDispo_LigneStock INT(5) NULL,
CONSTRAINT PRIMARY KEY(Reference_LigneStock)
)
ENGINE=INNODB;

```

```

CREATE TABLE Type_Mouvement
(
Reference_TypeMouvement SMALLINT(2) UNSIGNED NOT NULL AUTO_INCREMENT,
Libelle_TypeMouvement VARCHAR(30) NULL,
Sens_TypeMouvement ENUM('ENTREE','SORTIE'),
CONSTRAINT PRIMARY KEY(Reference_TypeMouvement)
)
ENGINE=INNODB;

```

```

CREATE TABLE Couleur
(
Reference_Couleur VARCHAR(10) NOT NULL,
Libelle_Couleur VARCHAR(30) NULL,
CONSTRAINT PRIMARY KEY(Reference_Couleur)
)
ENGINE=INNODB;

```

/ Ajout des clés étrangères dans la table 'Produit' */*

```

ALTER TABLE Produit ADD COLUMN Reference_Categorie VARCHAR(10) NULL;
ALTER TABLE Produit ADD CONSTRAINT FOREIGN KEY (Reference_Categorie)
REFERENCES Categorie(Reference_Categorie);
ALTER TABLE Produit ADD COLUMN Reference_Taille VARCHAR(10) NULL;
ALTER TABLE Produit ADD CONSTRAINT FOREIGN KEY (Reference_Taille) REFERENCES
Taille(Reference_Taille);
ALTER TABLE Produit ADD COLUMN Reference_Couleur VARCHAR(10) NULL;
ALTER TABLE Produit ADD CONSTRAINT FOREIGN KEY(Reference_Couleur) REFERENCES
Couleur(Reference_Couleur);
ALTER TABLE Produit ADD COLUMN Reference_Marque VARCHAR(10) NULL;
ALTER TABLE Produit ADD CONSTRAINT FOREIGN KEY(Reference_Marque) REFERENCES
Marque(Reference_Marque);
ALTER TABLE Produit ADD COLUMN Id_TVA SMALLINT(2) UNSIGNED NULL;
ALTER TABLE Produit ADD CONSTRAINT FOREIGN KEY(Id_TVA) REFERENCES
TVA(Id_TVA);

```

/ Ajout de la clé étrangère de la table 'Representant_Fournisseur' */*

```

ALTER TABLE Representant_Fournisseur ADD COLUMN Reference_Fournisseur VARCHAR(17)
NULL;
ALTER TABLE Representant_Fournisseur ADD CONSTRAINT FOREIGN KEY
(Reference_Fournisseur) REFERENCES Fournisseur(Reference_Fournisseur);
ALTER TABLE Representant_Fournisseur ADD COLUMN Sexe_Representant_Fournisseur
VARCHAR(10) NULL;
ALTER TABLE Representant_Fournisseur ADD CONSTRAINT FOREIGN KEY
(Sexe_Representant_Fournisseur) REFERENCES Sexe(Reference_Sexe);

```

/ Ajout de la clé étrangère de la table 'Sous_Catégorie' */*

```
ALTER TABLE Sous_Categorie ADD COLUMN Reference_Categorie VARCHAR(10) NOT NULL;  
ALTER TABLE Sous_Categorie ADD CONSTRAINT FOREIGN KEY (Reference_Categorie)  
REFERENCES Categorie(Reference_Categorie);
```

/ Ajout de clé étrangère de la table 'Commande' */*

```
ALTER TABLE Commande ADD COLUMN Reference_ModePaiement SMALLINT(2)  
UNSIGNED NULL;  
ALTER TABLE Commande ADD CONSTRAINT FOREIGN KEY (Reference_ModePaiement)  
REFERENCES Mode_Paiement(Reference_ModePaiement);  
ALTER TABLE Commande ADD COLUMN Reference_Client VARCHAR(20) NULL;  
ALTER TABLE Commande ADD CONSTRAINT FOREIGN KEY (Reference_Client)  
REFERENCES Client(Reference_Client);
```

/ Ajout de la clé étrangère de la table 'Ligne_Commande' */*

```
ALTER TABLE Ligne_Commande ADD COLUMN Reference_Commande VARCHAR(20) NOT NULL;  
ALTER TABLE Ligne_Commande ADD CONSTRAINT FOREIGN KEY (Reference_Commande)  
REFERENCES Commande(Reference_Commande);  
ALTER TABLE Ligne_Commande ADD COLUMN Id_Produit INT(10) UNSIGNED NOT NULL;  
ALTER TABLE Ligne_Commande ADD CONSTRAINT FOREIGN KEY (Id_Produit)  
REFERENCES Produit(Id_Produit);
```

/ Ajout des clés étrangères de la table 'Ligne_Stock' */*

```
ALTER TABLE Ligne_Stock ADD COLUMN Reference_TypeMouvement SMALLINT(2)  
UNSIGNED NOT NULL;  
ALTER TABLE Ligne_Stock ADD CONSTRAINT FOREIGN KEY (Reference_TypeMouvement)  
REFERENCES Type_Mouvement(Reference_TypeMouvement);  
ALTER TABLE Ligne_Stock ADD COLUMN Id_Produit INT(10) UNSIGNED NOT NULL;  
ALTER TABLE Ligne_Stock ADD CONSTRAINT FOREIGN KEY (ID_Produit) REFERENCES  
Produit(Id_Produit);
```

/ Ajout de la clé étrangère de la table 'Client' */*

```
ALTER TABLE Client ADD COLUMN CP_Client VARCHAR(5) NULL;  
ALTER TABLE Client ADD COLUMN Ville_Client VARCHAR(40) NULL;  
ALTER TABLE Client ADD CONSTRAINT FOREIGN KEY (CP_Client, Ville_Client)  
REFERENCES Code_Postal(CP_CodePostal, Ville_CodePostal);
```

/ Ajout de la clé étrangère de la table 'Fournisseur' */*

```
ALTER TABLE Fournisseur ADD COLUMN CP_Fournisseur VARCHAR(5) NULL;  
ALTER TABLE Fournisseur ADD COLUMN Ville_Fournisseur VARCHAR(40) NULL;  
ALTER TABLE Fournisseur ADD CONSTRAINT FOREIGN KEY (CP_Fournisseur,  
Ville_Fournisseur) REFERENCES Code_Postal(CP_CodePostal, Ville_CodePostal);
```

/ Ajout de la clé étrangère de la table 'Approvisionnement' */*

```
ALTER TABLE Approvisionnement ADD COLUMN Reference_Fournisseur VARCHAR(17)  
NULL;  
ALTER TABLE Approvisionnement ADD CONSTRAINT FOREIGN KEY (Reference_Fournisseur)  
REFERENCES Fournisseur(Reference_Fournisseur);
```

/ Ajout des clés étrangères de la table 'Ligne_Approvisionnement */*

```
ALTER TABLE Ligne_Approvisionnement ADD COLUMN Reference_Approvisionnement  
VARCHAR(20) NOT NULL;
```

```
ALTER TABLE Ligne_Approvisionnement ADD CONSTRAINT FOREIGN  
KEY(Reference_Approvisionnement) REFERENCES  
Approvisionnement(Reference_Approvisionnement);
```

```
ALTER TABLE Ligne_Approvisionnement ADD COLUMN Id_Produit INT(10) UNSIGNED NOT  
NULL;
```

```
ALTER TABLE Ligne_Approvisionnement ADD CONSTRAINT FOREIGN KEY(Id_Produit)  
REFERENCES Produit(Id_Produit);
```

Modélisation UML

Diagramme de contexte

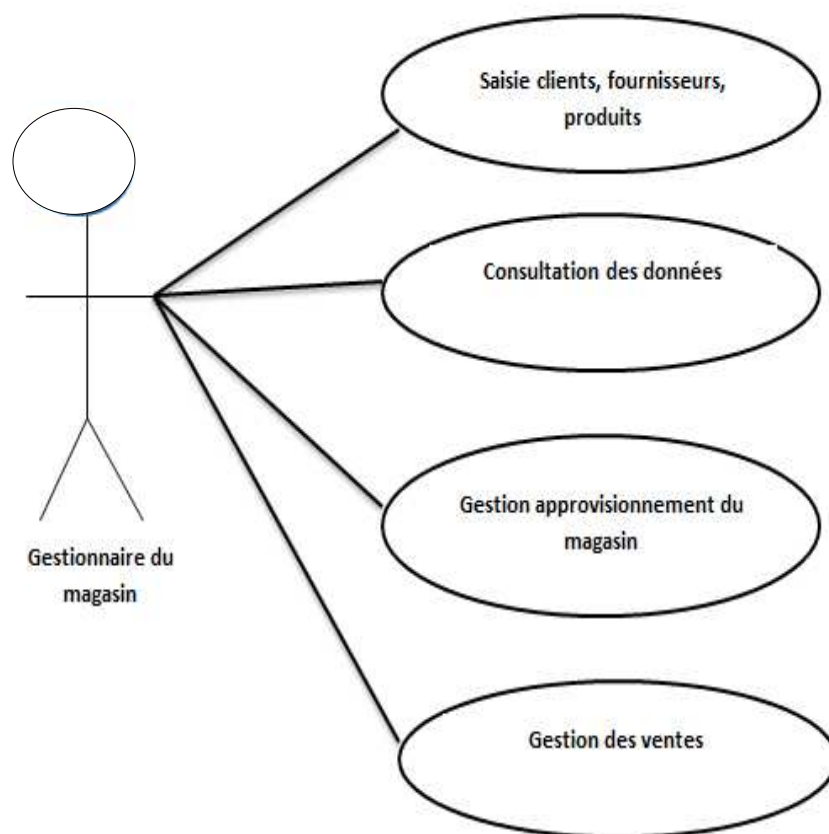
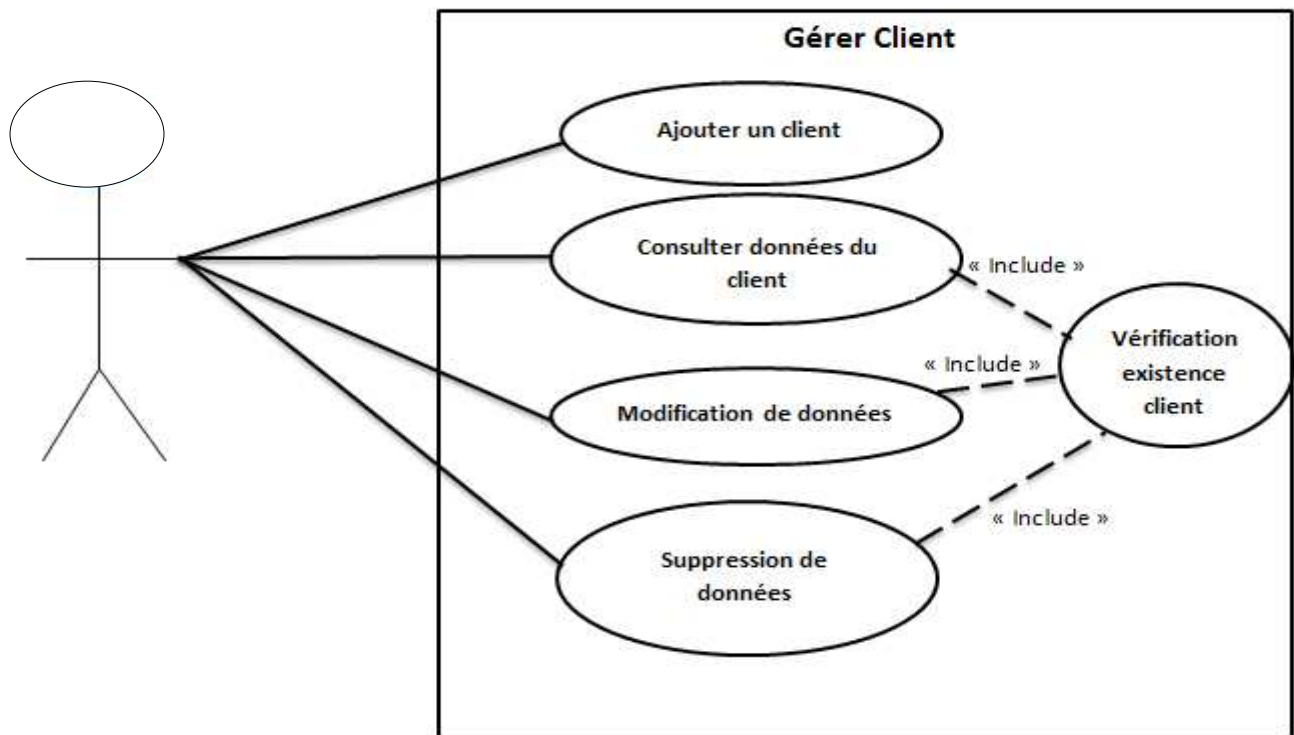


Diagramme de cas d'utilisation



Objectif:

Ce cas d'utilisation permet d'ajouter un client dans la base de données en renseignant ses coordonnées, l'acteur pourra aussi consulter et modifier les informations d'un client. Un client pourra être supprimé. L'acteur principal est le gestionnaire du magasin, il s'occupe des ventes, de la gestion du stock, des clients et des fournisseurs lui seul pourra utiliser l'application. Ce cas d'utilisation ne gère aucun acteur secondaire.

Préconditions:

Pour que le cas d'utilisation puisse être déclenché, il faut que l'utilisateur soit connecté à l'application et à la base de données et que celle-ci soit bien disponible.

Scénarii:

- Quand le gestionnaire du magasin rencontre un nouveau client qui fait un achat dans son magasin, il lui demande si cette personne veut devenir adhérent à son magasin, si le client répond positivement, le gestionnaire peut par le biais d'un formulaire ajouter cette personne dans la base de données en insérant ses principales coordonnées : Nom, Prénom, Adresse, Ville, Code Postal, Adresse mail, numéros de téléphone.

- Si le gestionnaire du magasin veut vérifier l'existence d'un client ou consulter les données d'un client, il a un champ recherche à sa disposition pour retrouver un client, sur celui-ci elle pourra retrouver les coordonnées principales qu'elle a renseigné à l'inscription, sa référence de client et sa date d'inscription.

- Si un client se présente et demande une modification de ses données dans la base de données car il a déménagé ou qu'il a changé d'adresse mail et qu'il ne reçoit plus les informations du magasin, le gestionnaire du magasin peut modifier ses informations grâce à un formulaire prévu à cet effet.

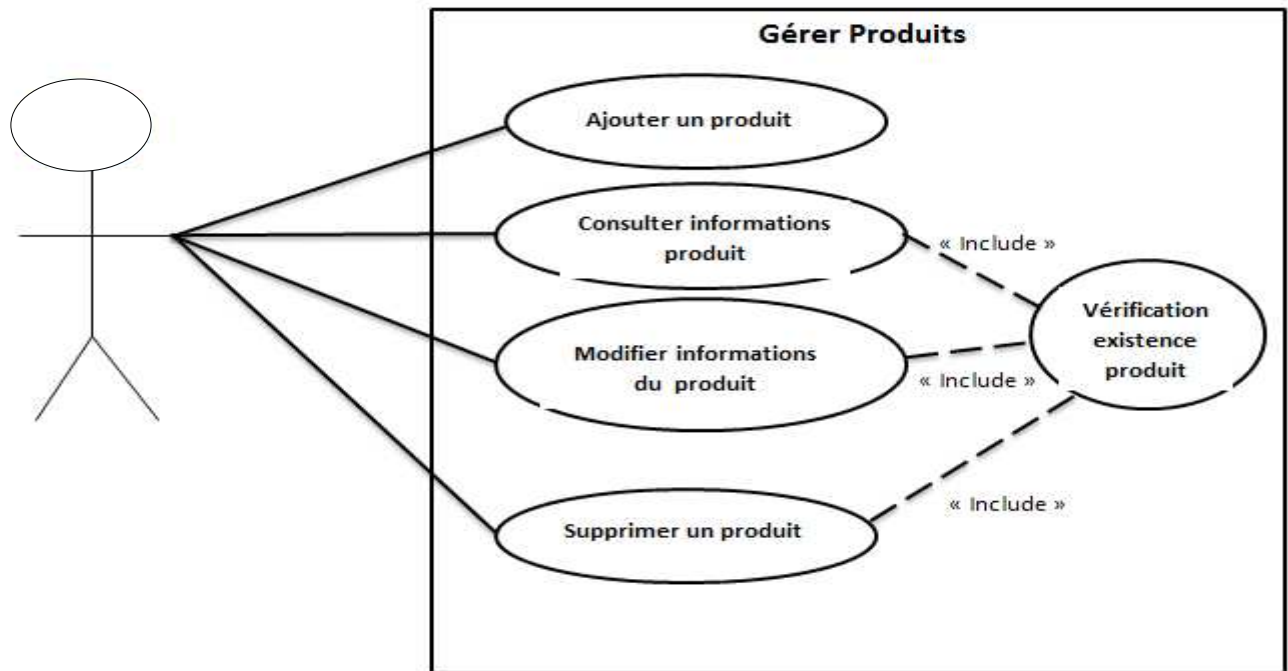
- Si un client ne veut plus adhérer au magasin, il peut demander au magasin de le supprimer de sa base de données, l'utilisateur pourra donc le supprimer de la base de données grâce à un lien de suppression.

Post conditions :

Après les actions Ajouter, Modifier, Supprimer, la base de données sera mise à jour, et le cas d'utilisation sera à nouveau opérationnel. Après l'action de consultation de données, le système ne changera pas.

Date: 03/12/2012

Responsables : GENEY - TARNAUD



Objectif:

L'acteur principal pourra ajouter, modifier ou supprimer un produit dans la base de données. Une fois le produit dans la base de données, l'utilisateur pourra consulter les informations des produits.

Préconditions:

Pour que le cas d'utilisation puisse être déclenché, il faut que l'utilisateur soit connecté à l'application et à la base de données et que celle-ci soit bien disponible.

Scénarii:

- Si le magasin est approvisionné avec des nouveaux produits le gestionnaire du magasin peut ajouter à l'aide d'un formulaire le produit dans la base de données en renseignant sa référence, son libellé, sa taille, sa couleur, sa marque, sa catégorie, sous-catégorie et sa description.
- Si lors de la saisie d'un produit la description est mal formulée ou possède une erreur, l'utilisateur pourra aller modifier cette description à l'aide d'un formulaire de modification.
- Si un client demande au gestionnaire du magasin s'il possède un produit en une taille différent, il pourra rechercher ce produit grâce à sa référence et vérifier la disponibilité de ce produit avec ses différentes informations.
- Si un produit n'est plus en stock et qu'il ne sera plus jamais recommandé, l'utilisateur peut aller supprimer ce produit de la base de données.

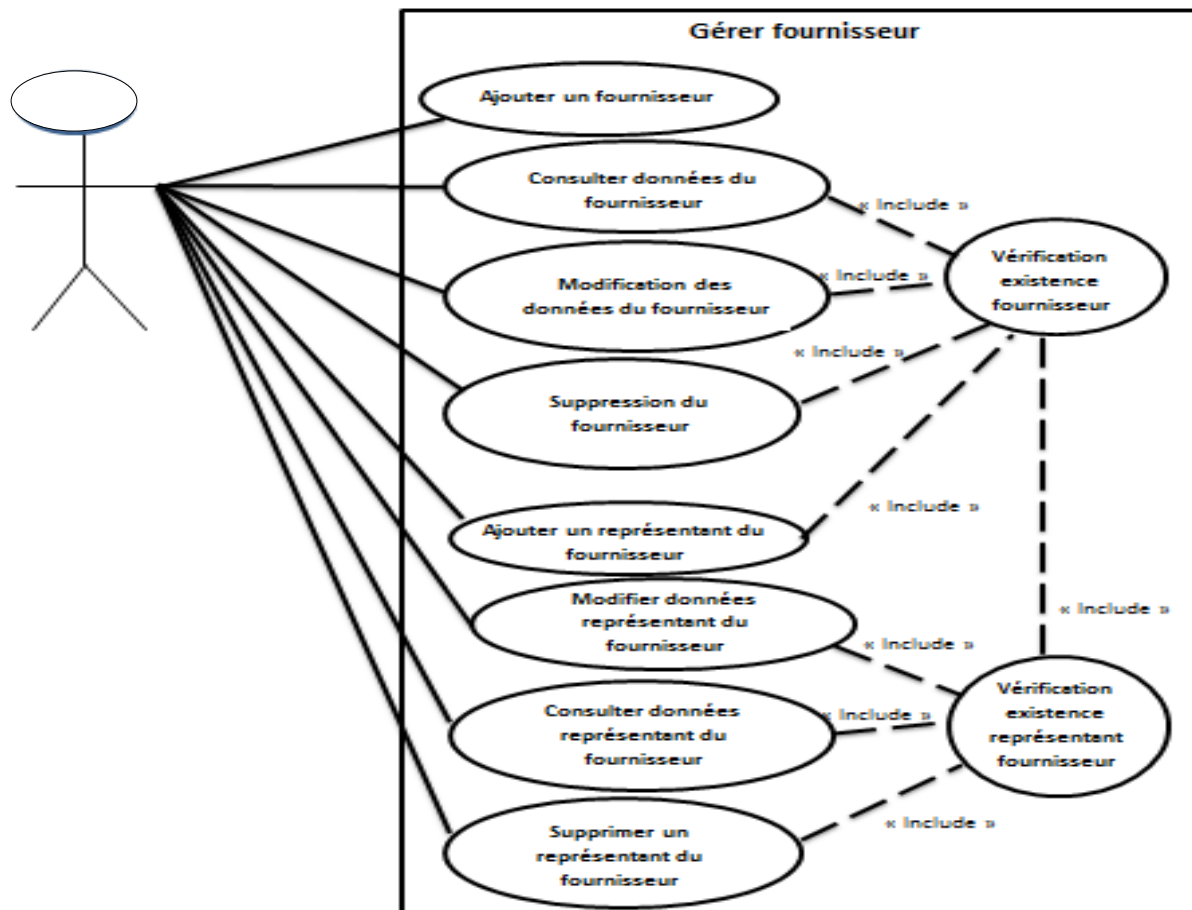
Post conditions:

Après les actions Ajouter, Modifier, Supprimer, la base de données sera mise à jour, et le cas d'utilisation sera à nouveau opérationnel. Après l'action de consultation de données, le système ne changera pas.

Date: 03/12/2012

Responsables: GENEY - TARNAUD

Version : 1.0



Objectif:

L'acteur principal pourra ajouter, modifier ou supprimer une facture lors d'une vente. L'utilisateur pourra aussi ajouter une quantité de produit dans le stock.

Préconditions:

Pour que le cas d'utilisation puisse être déclenché, il faut que l'utilisateur soit connecté à l'application et à la base de données et que celle-ci soit bien disponible.

Scénarii:

- Si le magasin reçoit une commande de produit qu'elle a déjà enregistré dans la base de données, l'utilisateur peut ajouter le nombre de produit qu'elle a reçu, ce qui recalculera le nombre de ce produit dans le stock.
- Lors d'une vente, le vendeur crée une facture avec le produit et le prix de vente, pour cela il devra renseigner la référence du produit et une remise s'il y en a une.
- Si une facture est fautive, qu'elle porte sur le mauvais produit, le vendeur peut modifier sa facture qu'il vient de créer.
- Si un produit revient après une vente dû à un problème sur le produit et qu'il ne peut être échangé, le gestionnaire peut supprimer la facture qui a été créée pour éviter d'avoir des erreurs sur le nombre de vente et les bénéfices engendrés.

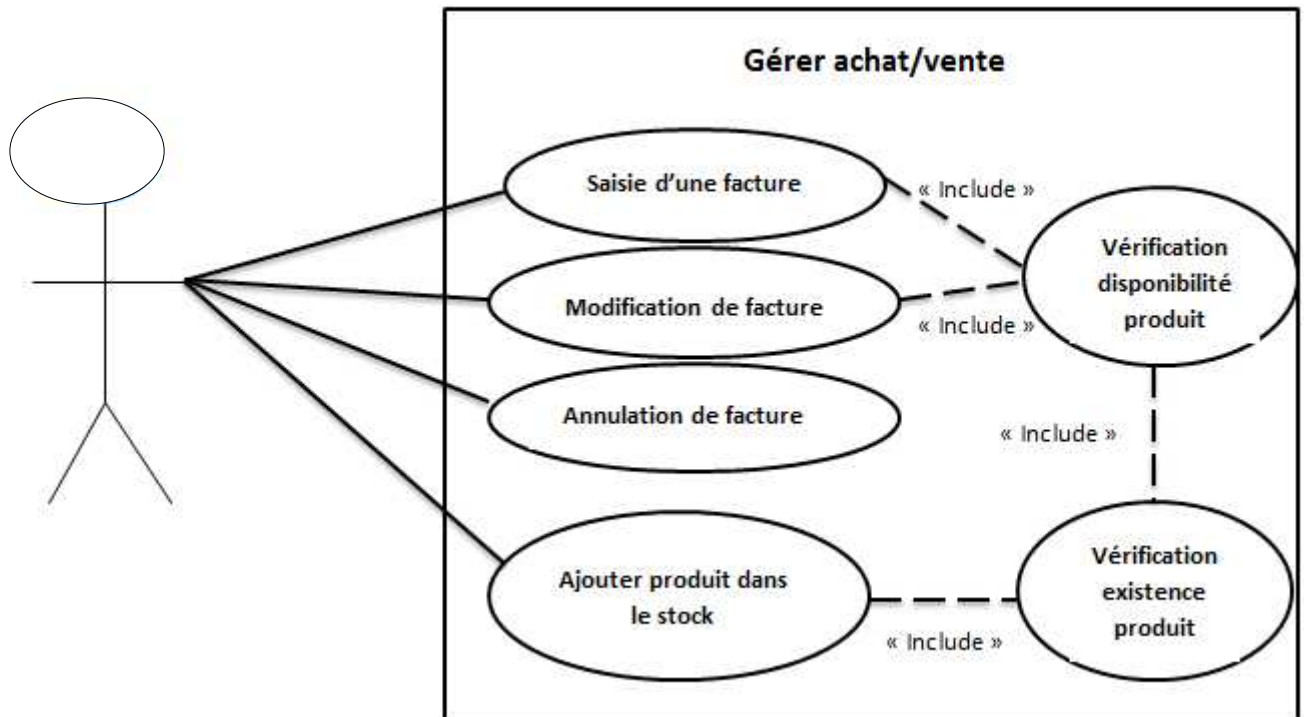
Post conditions:

Après les actions Ajouter, Modifier, Supprimer, la base de données sera mise à jour, et le cas d'utilisation sera à nouveau opérationnel. Après l'action de consultation de données, le système ne changera pas.

Date: 03/12/2012

Responsables: GENEY - TARNAUD

Version: 1.0



Objectif:

L'acteur principal pourra ajouter, modifier ou supprimer une facture lors d'une vente. L'utilisateur pourra aussi ajouter une quantité de produit dans le stock.

Préconditions:

Pour que le cas d'utilisation puisse être déclenché, il faut que l'utilisateur soit connecté à la base de données et que des produits soient bien enregistrés dans la base de données.

Scénarii:

- Si le magasin reçoit une commande de produit qu'elle a déjà enregistré dans la base de données, l'utilisateur peut ajouter le nombre de produit qu'elle a reçu, ce qui recalculera le nombre de ces produits dans le stock.
- Lors d'une vente, le vendeur crée une facture avec le produit et le prix de vente, pour cela il devra renseigner la référence du produit et une remise s'il y en a une.
- Si une facture est fautive, qu'elle porte sur le mauvais produit, le vendeur peut modifier sa facture qu'il vient de créer.
- Si un produit revient après une vente dû à un problème sur le produit et qu'il ne peut être échangé, le gestionnaire peut supprimer la facture qui a été créée pour éviter d'avoir des erreurs sur le nombre de vente et les bénéfices engendrés.

Post conditions:

Après les actions Ajouter, Modifier, Supprimer, la base de données sera mise à jour, et le cas d'utilisation sera à nouveau opérationnel. Après l'action de consultation de données, le système ne changera pas.

Date: 03/12/2012

Responsables: GENEY - TARNAUD

Version: 1.0

Diagrammes de séquences

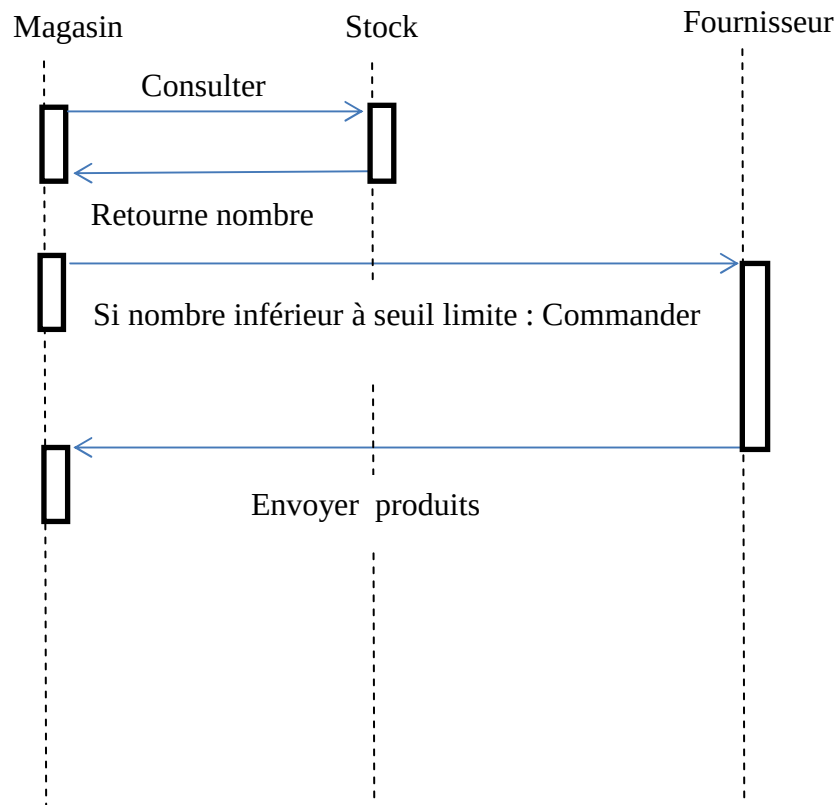
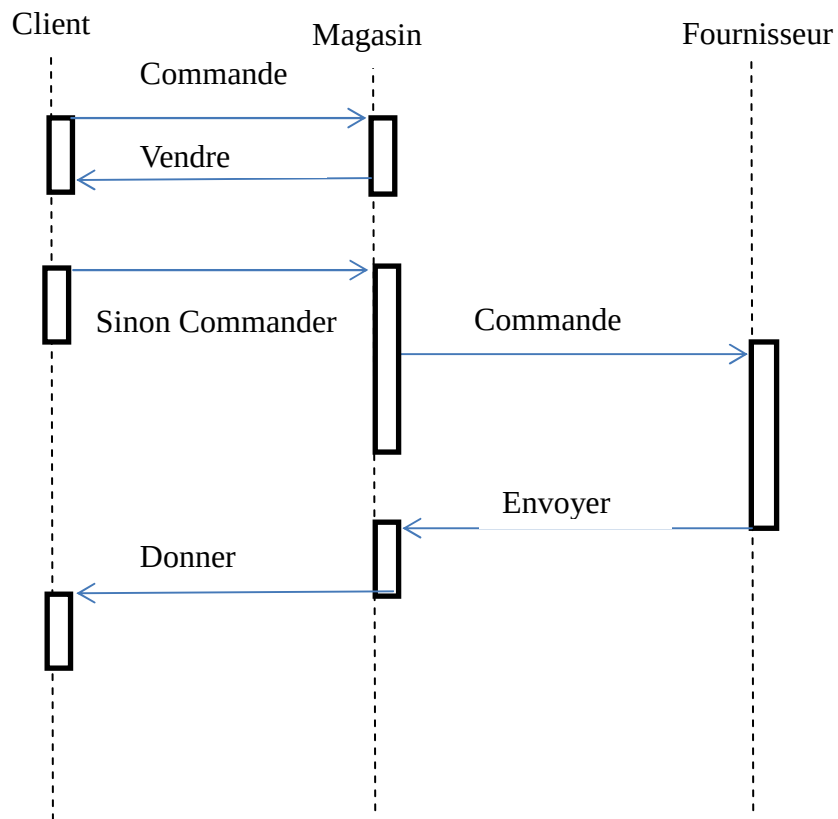
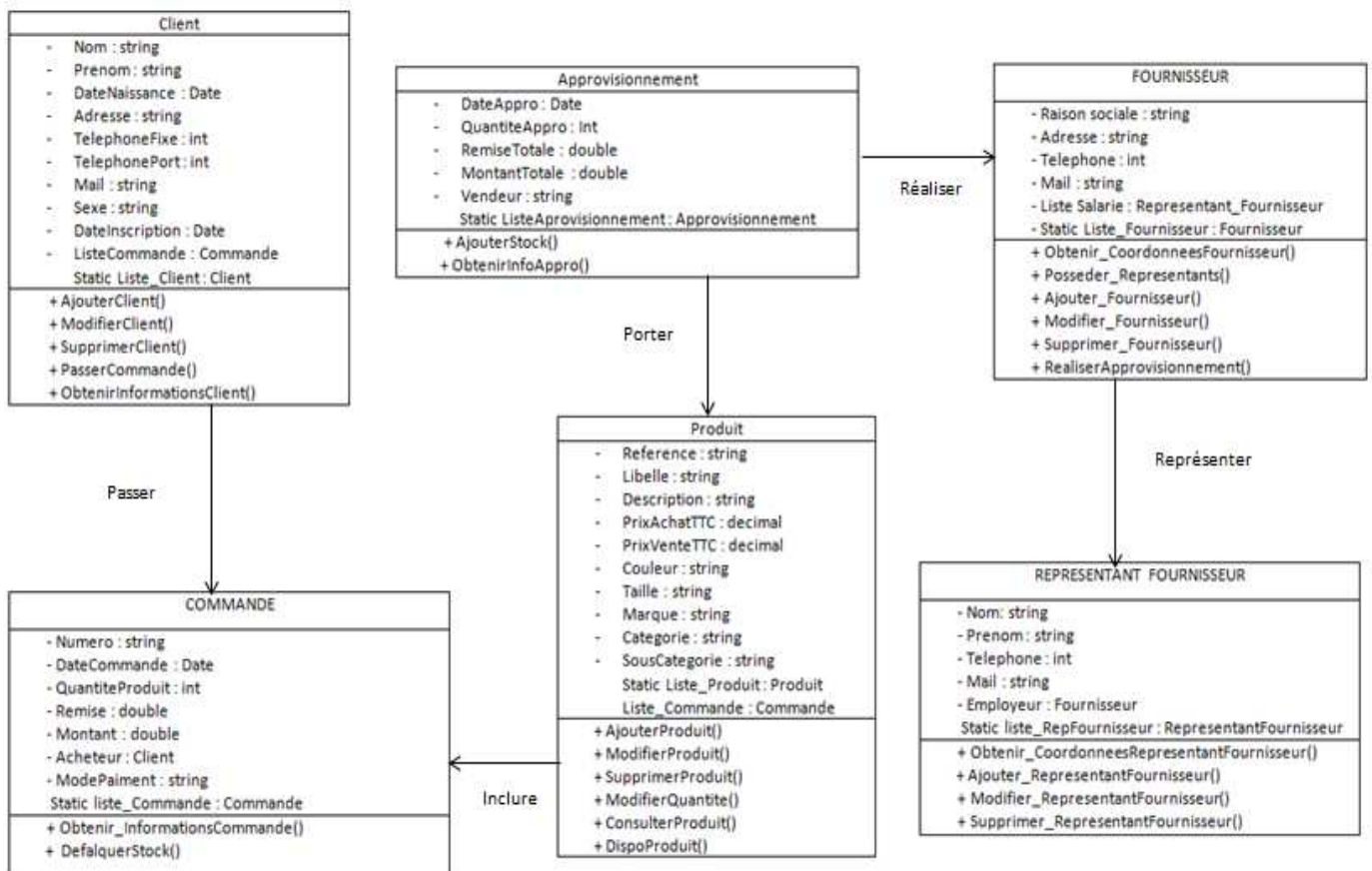


Diagramme de classes



Introduction:

Pour développer cette application nous avons suivi le cahier des charges établit avec le client. L'ergonomie de l'application a été pensée pour que l'utilisateur puisse exécuter les actions définies dans le cahier des charges sans difficulté et avec rapidité. Les onglets sont triés par ordre d'utilisations.

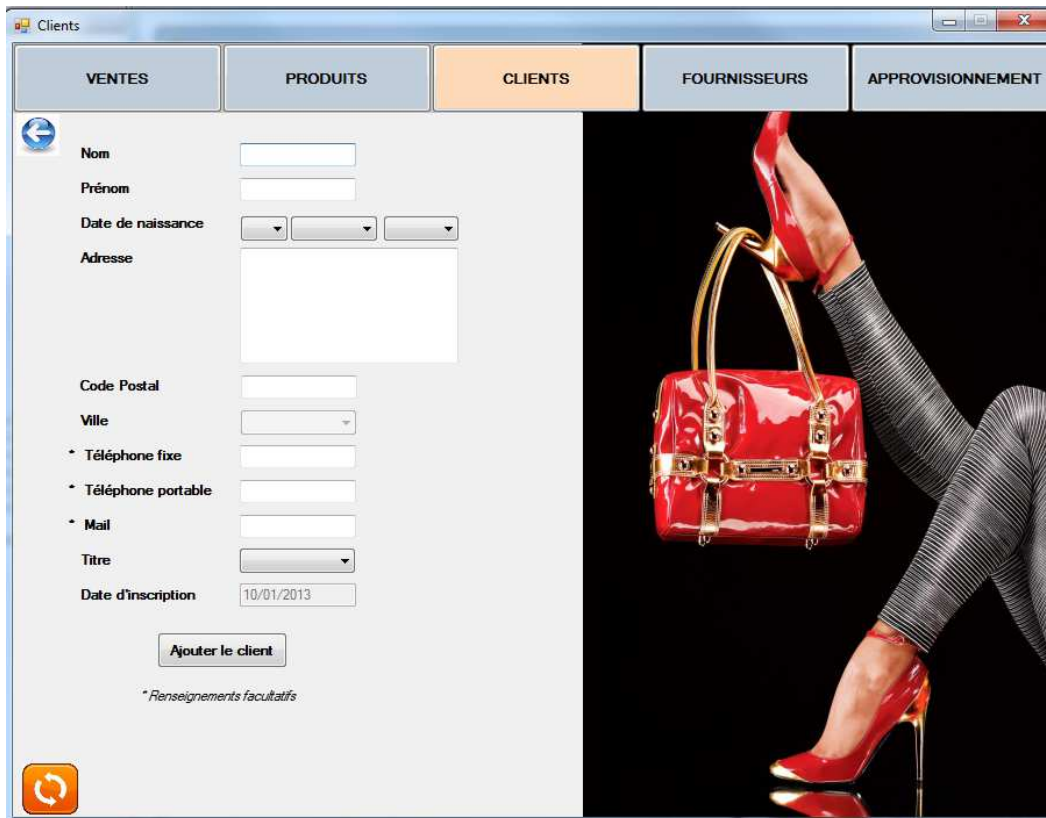
Ouverture du programme :

La première interface est une fenêtre permettant de s'identifier pour accéder à l'application et de créer la connexion à la base de données.

Menu Client :

Cette page permet de choisir une action concernant un client : Ajouter un client amènera vers un formulaire d'ajout de client. Obtenir les informations d'un client amènera vers un formulaire de consultation et de modification du client.

Ajouter un Client :



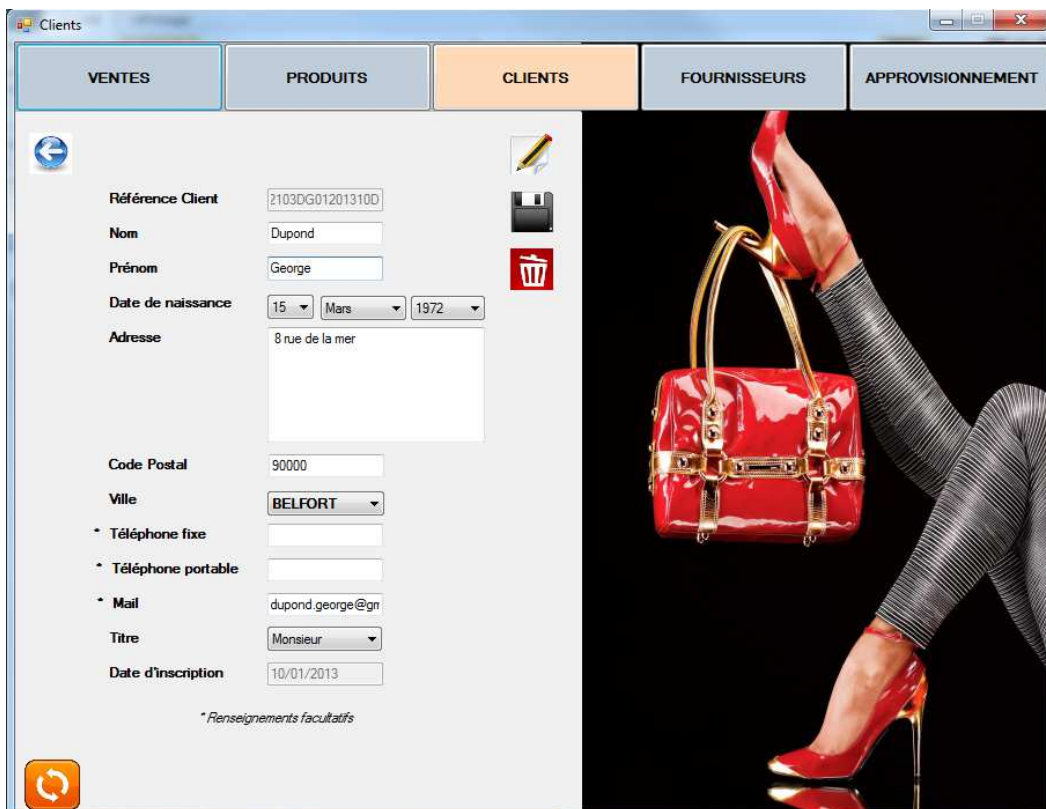
The screenshot shows the 'Clients' application window with the 'CLIENTS' tab selected. The form on the left contains the following fields:

- Nom:
- Prénom:
- Date de naissance:
- Adresse:
- Code Postal:
- Ville:
- * Téléphone fixe:
- * Téléphone portable:
- * Mail:
- Titre:
- Date d'inscription: 10/01/2013

At the bottom of the form is a button labeled 'Ajouter le client' and a note: '* Renseignements facultatifs'. On the right side of the window is a large image of a woman's legs in high heels holding a red handbag.

Ce formulaire permet d'ajouter un client dans la base de données en renseignant ses informations telles que son nom, son prénom, sa date de naissance, son adresse (rue, code postal, ville). Le numéro de téléphone et l'adresse mail sont facultatifs. Renseigner si c'est un homme ou une femme. Le bouton « Ajouter le client » enregistre le client dans la base de données.

Consultation et modification client :



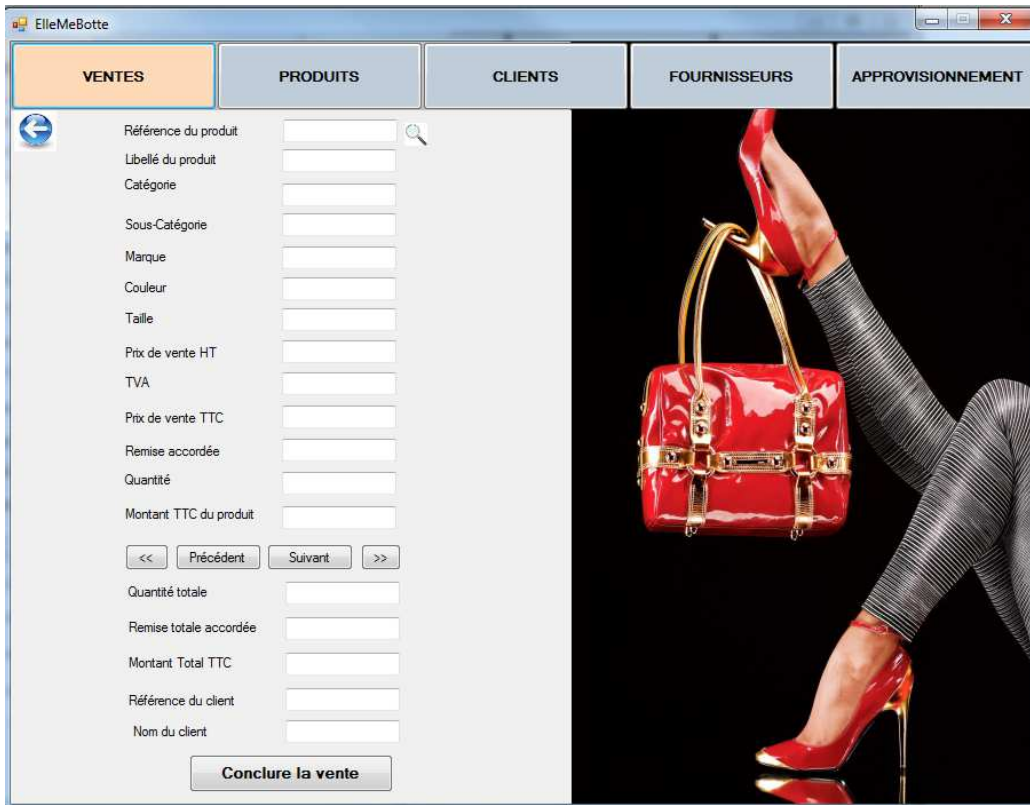
The screenshot shows the 'Clients' application window with the 'CLIENTS' tab selected. The form on the left contains the following fields:

- Référence Client: 2103DG012013100
- Nom: Dupond
- Prénom: George
- Date de naissance: 15 Mars 1972
- Adresse: 8 rue de la mer
- Code Postal: 90000
- Ville: BELFORT
- * Téléphone fixe:
- * Téléphone portable:
- * Mail: dupond.george@gn
- Titre: Monsieur
- Date d'inscription: 10/01/2013

At the bottom of the form is a note: '* Renseignements facultatifs'. On the right side of the window is a large image of a woman's legs in high heels holding a red handbag.

Consultation des données d'un client, toutes ses informations sont visibles. L'utilisateur peut toutes les modifier à l'exception de la référence du client et de la date d'inscription du client. L'utilisateur peut aussi supprimer le client.

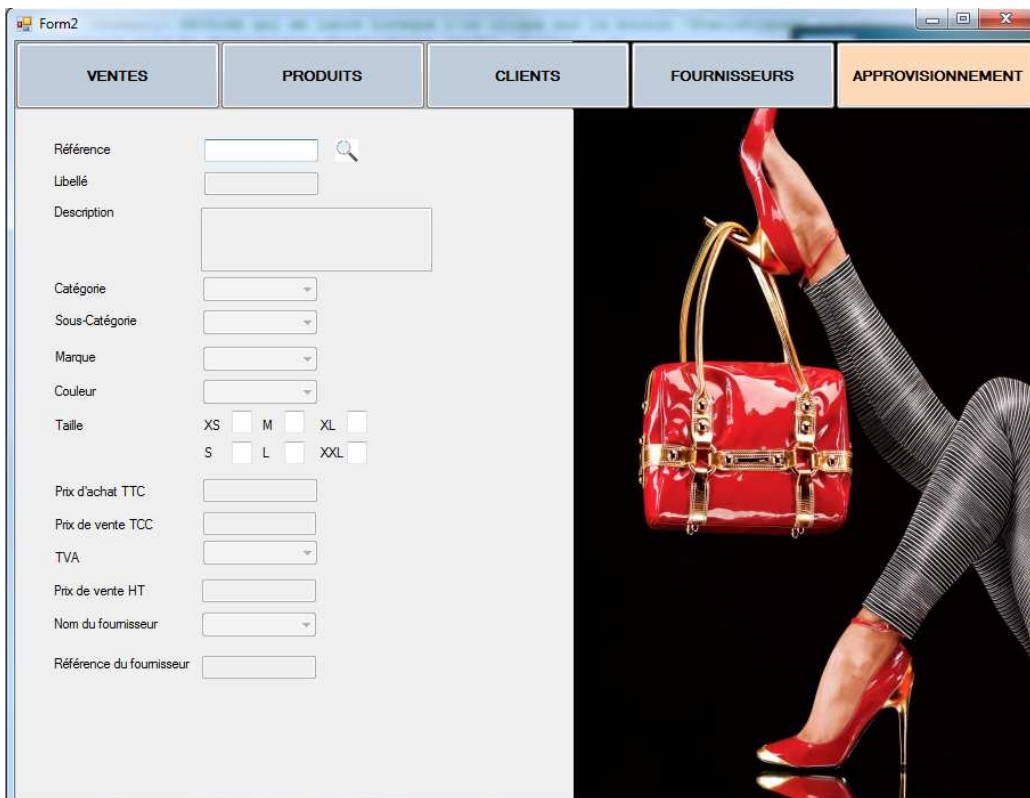
Effectuer une vente :



Pour effectuer une vente, on recherche le produit par sa référence, les informations correspondantes aux produits se remplissent automatiquement, l'utilisateur peut accorder une remise au client, ce qui calculera avec la quantité, le prix de vente Hors Taxe et la TVA le montant TTC du produit. Les boutons précédant et suivant permettent de renseigner un autre produit pour la vente. La quantité totale est ensuite calculée ainsi que la

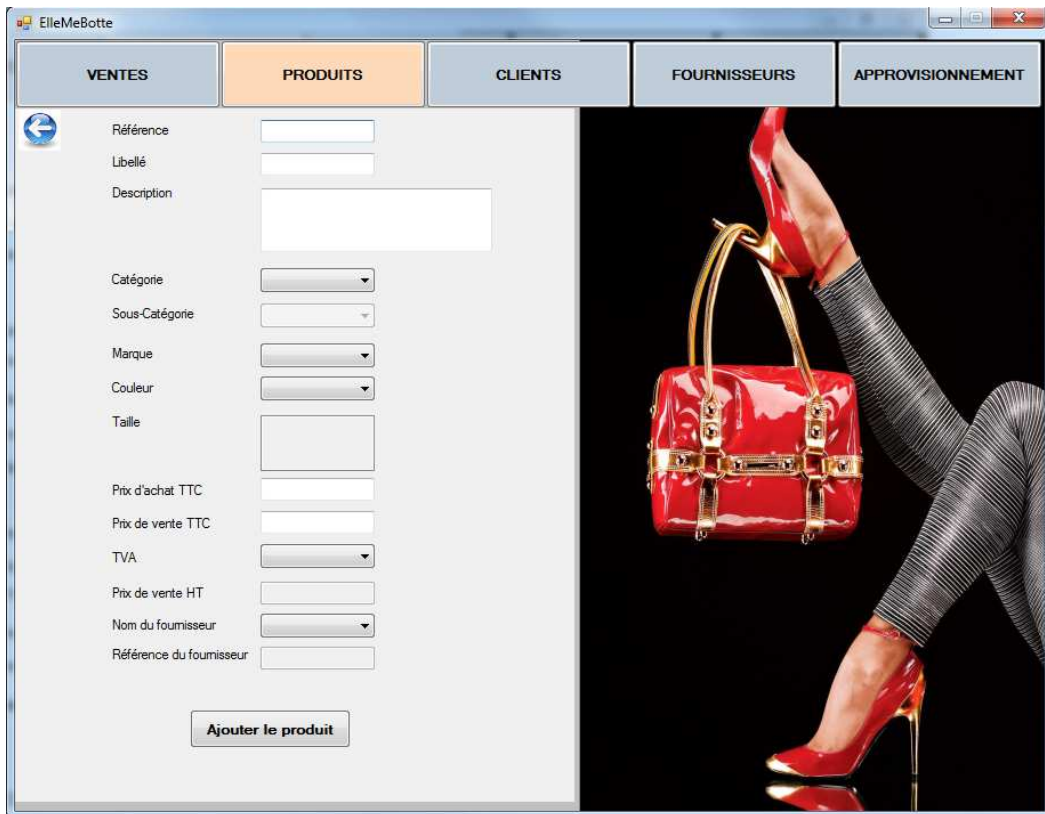
remise totale et le montant total TTC. Pour conclure la vente il faut renseigner la référence et le nom du client puis cliquer sur le bouton « Conclure la vente » ce qui enregistrera cette vente dans la base de données et mettra à jour les stocks.

Approvisionnement :



Dans le module approvisionnement, l'utilisateur entre une référence d'un produit, celle-ci permet de remplir automatiquement les champs grisés, les textBox à côté de la taille permettent de renseigner la quantité à ajouter. Pour finir il faudra ajouter le nom du fournisseur.

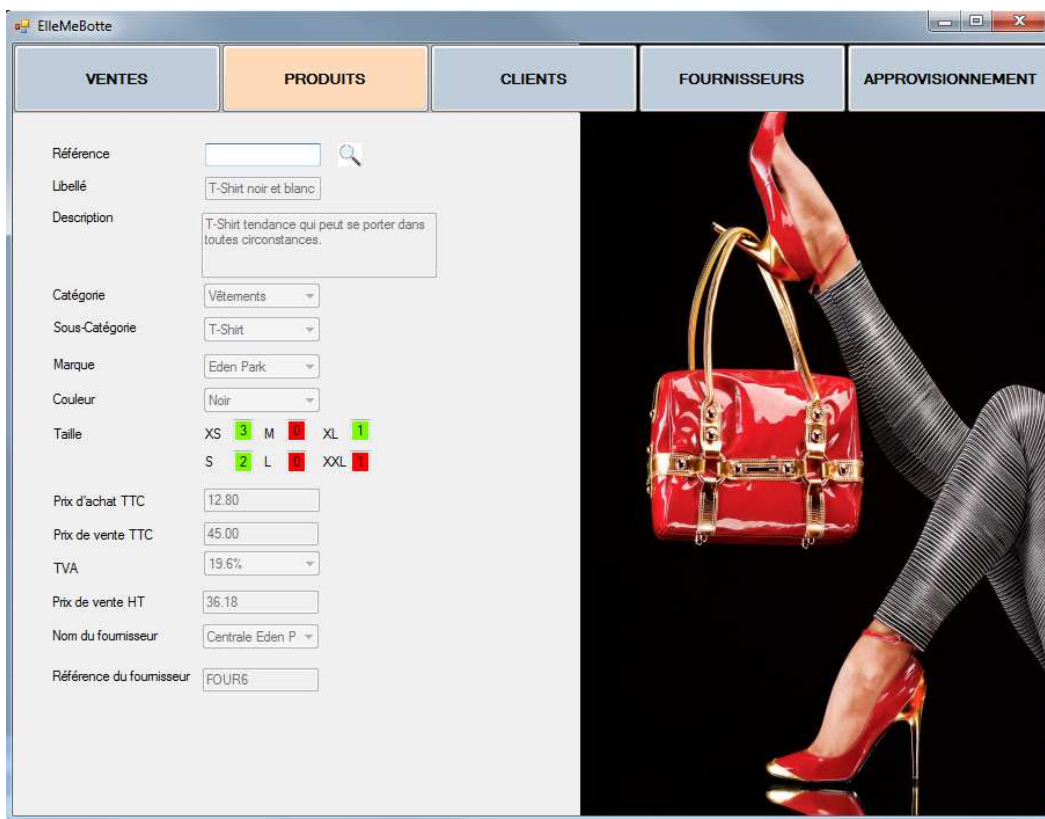
Ajout d'un produit :



Ajout d'un produit, en y renseignant une référence, un libellé, une description, une catégorie qui fera accéder à une deuxième zone de liste pour choisir la sous-catégorie correspondante, il faut aussi choisir une marque, une couleur, une taille, un prix d'achat TTC et un prix de vente TTC avec la TVA correspondante, le prix de vente HT sera calculé, il faudra choisir un nom de fournisseur pour savoir qui a vendu le produit au

magasin. Le bouton ajouter le produit l'ajoutera dans la base de données.

Consultation d'un produit :



Formulaire de consultation d'un produit avec toutes les données enregistrées lors de l'ajout du produit avec en plus la disponibilité ou non par taille.

Principales requêtes développées

Voici la liste des requêtes les plus utilisées dans l'application :

1. Connaître toutes les informations d'un produit à partir d'une donnée saisie par l'utilisateur

```
SELECT Libelle_Produit, Description_Produit, Reference_Marque, Reference_Taille,  
Reference_SousCategorie, Id_Tva, Reference_Couleur FROM PRODUIT WHERE Reference_Produit =  
'G50454' ;
```

2. Supprimer un client de la base de données

```
DELETE FROM CLIENT WHERE Reference_Client = 'CLT545455GD5454C' ;
```

3. Insérer un nouveau produit dans la base de données

```
INSERT INTO PRODUIT (Reference_Produit, Libelle_Produit, Description_Produit, Reference_Marque,  
Reference_Taille, Reference_SousCategorie, Id_Tva, Reference_Couleur) VALUES ('GSF502', 'Chaussure  
de mode italienne', 'Très belle chaussure tendance pour l'été qui sera s'adaptée à toute vos envies  
vestimentaires', 'MARQ2', '39', 'SCAT1', 2, 'Noire') ;
```

4. Récupérer toutes les villes par rapport au code postal sélectionné

```
SELECT Ville_CodePostal FROM Code_Postal WHERE CP_CodePostal = '90000' ;
```

5. Récupérer uniquement les tailles disponibles en stock d'un produit

```
SELECT Reference_Taille FROM PRODUIT, LIGNE_STOCK WHERE PRODUIT.Id_Produit =  
LIGNE_STOCK.Id_Produit AND QuantiteDispo_LigneStock > 0 AND Reference_Produit = 'GFM510' ;
```

6. Mise à jour des informations d'un client

```
UPDATE CLIENT SET Nom_Client = 'DUPONT' WHERE Reference_Client = 'CLT44545454RD54D' ;
```

Quelques exemples de code

1. Connexion et déconnexion à la base de données

```
///
```

2. Lancement d'un thread qui tourne en arrière-plan et qui permet de savoir à l'instant T si la connexion à la base de données est bonne ou non

```
#region THREAD CONNEXION BDD

///
```

3. Méthode d'insertion d'un produit dans la base de données

```
int i = 0;
MySQLCommand mysql_Ajout_Prod = new MySqlCommand();

// On ajoute les champs dans la base de données
// On prépare les requêtes pour récupérer les références de : Sous-Catégorie, Marque et Tva
string w_ReqRefSCat = "SELECT Reference_SousCategorie FROM SOUS_CATEGORIE WHERE Libelle_SousCategorie = '" + w_SousCategorie + "'";
string w_ReqRefMarque = "SELECT Reference_Marque FROM MARQUE WHERE Libelle_Marque = '" + w_Marque + "'";
string w_ReqIdTva = "SELECT Id_Tva FROM TVA WHERE Taux_Tva = '" + w_Tva + "'";

// On va boucler sur les tailles et ajouter autant de lignes de produit qu'il y a de tailles
for (i = 0; i < lst_Taille.Count; i++)
{
    mysql_Ajout_Prod.CommandText = @"INSERT INTO Produit(Reference_Produit, Libelle_Produit, Description_Produit, PrixAchatTTC_Produit, " +
        "PrixVenteTTC_Produit, Reference_SousCategorie, Reference_Taille, Reference_Marque," +
        "Id_TVA) VALUES ('" + w_Reference + "','" + w_Libelle + "','" + w_Description + "','" + w_Pattc + "','" + w_Pvttc +
        "', (" + w_ReqRefSCat + "),'", lst_Taille[i].ToString() + "','" + w_ReqRefMarque + "'), (" + w_ReqIdTva + "));";
    mysql_Ajout_Prod.Connection = sp_fonctions.P_mysql_CheminConnexion;
    mysql_Ajout_Prod.ExecuteNonQuery();

    // On ajoute la couleur du produits dans la table 'Colorer'
    mysql_Ajout_Prod.CommandText = "INSERT INTO Colorer VALUES ('" + w_Couleur + "','" +
        "(SELECT MAX(Id_Produit) FROM PRODUIT WHERE Reference_Produit = '" + w_Reference + "')';";
    mysql_Ajout_Prod.ExecuteNonQuery();
}

// On vide les textbox
sp_fonctions.sp_ViderTextboxPanel(ref this.pn_AjoutProd);

// On sélectionne les premiers indexs de chaque combobox
this.cb_CatProd.SelectedIndex = 0;
this.cb_MarqueProd.SelectedIndex = 0;
this.cb_CouleurProd.SelectedIndex = 0;
this.cb_TvaProd.SelectedIndex = 0;
this.cb_NomFourProd.SelectedIndex = 0;
```

Conclusion

La réalisation de ce projet nous a permis d'apprendre plusieurs choses, étant à deux sur ce projet, nous avons dû travailler en équipe. Ce projet nous a permis de nous familiariser avec le milieu commercial, nous avons analysé la gestion du stock dans son ensemble en partant de l'approvisionnement du produit jusqu'à sa vente. De ce fait nous avons pu identifier les différents critères entrant en compte tels que le calcul de la Tva, le référencement des produits, les mouvements de produits... Ce projet qui est réel et concret a augmenté notre motivation et notre concentration, nous avons pu être rigoureux et très professionnel afin de fournir un logiciel adapté au marché du client. L'application a été pensée pour traiter la majorité des cas d'utilisation possibles et pour prévenir des éventuels problèmes rencontrés lors de la prise en main. Nous dégageons de cette expérience un énorme point positif car elle a enrichi nos compétences personnelles et plus précisément professionnelles. Cependant, si cela était à refaire nous consacrerions beaucoup plus de temps à l'analyse du projet car elle permet d'éviter de multiples refontes de codes lors de la phase de développement.